CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

A source-code diffing tool needs an accurate, efficient way to map one abstract syntax tree (AST) onto another. Decades of research have given the field exact tree-edit-distance algorithms, Zhang-Shasha and APTED, and source-code-specific heuristics, GumTree, plus a newer generation of practical, tree-sitter-based tools, difftastic and diffsitter. This paper presents CodeDiff, a syntax-aware code-diffing tool that builds directly on this line of work, combining APTED's exact tree edit distance, applied to bounded subtree pairs rather than to whole files, with GumTree's move-recovery heuristic and Myers' O() sequence diff into one five-phase matching pipeline. CodeDiff's own contribution is not a new matching algorithm but two engineering disciplines applied on top of this foundation. First, its performance and robustness targets come from an empirical study of 100 real-world open-source repositories, about 1.35 million files in 30 languages, not from worst-case asymptotic analysis alone. Second, each optional heuristic pass ships only if an ablation study shows a measurable accuracy gain on a corpus of 468 real-world diffs with human-verified ground truth, so that a technique's published benefit is confirmed in CodeDiff's own pipeline rather than assumed to transfer unchanged. On that same ground-truth corpus, CodeDiff maps 99.94% of AST nodes correctly, and reaches that accuracy at a speed and reliability suited to everyday, interactive use—read alongside GumTree, difftastic, and diffsitter not as a replacement for any of them, but as one way of combining what each already does well. The corpus also answers a question about evaluation itself: in 15.3% of the changes annotated with the facility to record it, no single node mapping is the correct one, so any accuracy metric scored against one fixed pairing can record correct output as wrong on a sixth of them.

Conference'26, June 03–05, 2026, Ludbreg, Croatia

ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**; *Empirical software validation*.

KEYWORDS

source code differencing, syntax-aware diffing, empirical software engineering, source code tree edit distance

ACM Reference Format:
Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference'26).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Code diffing, the process of comparing two pieces of source code and computing the set of operations that transforms one into the other, underlies modern version control and, especially, modern code review. During a review, developers read the diff to see what changed and what stayed the same, so the quality of the diff directly shapes the quality of the review.

The standard diff used by almost all developers and platforms today is line-based: the file is treated as a sequence of text lines, and the tool computes a shortest edit script between the two sequences using three operations, insert, delete, and update. This approach is computationally cheap and fast even on modest hardware. However, most implementations offer no move operation at all, and the tools that do support one support it only in limited ways. A whole class of common changes—extracting code into a reusable function, wrapping a block in a condition, reordering declarations—is ultimately a relocation of existing code, and without a move operation every such change must be rendered as a full deletion plus a full insertion. Reconstructing the correspondence between the deleted and inserted halves is then left to the reviewer, by eye, which is exactly the kind of mechanical, attention-taxing work humans do least reliably.

One way to compute diffs with a genuine move operation is to compare the code's abstract syntax tree (AST) instead of its raw text. An AST diff maps nodes of one tree onto nodes of the other; a move is a mapping whose nodes match but whose ancestry changed. Tree diffing is a well-studied field, and Section 2 surveys the work this paper builds on. In practice, however, tree diffing has seen little adoption, and we argue the central reason is cost: the best exact tree-diffing algorithms are $O(n^3)$ in tree size, and Section 3 measures

directly how often a whole-tree computation at real-world file sizes exceeds an interactive time budget.

This paper presents CodeDiff, a syntax-aware code-diffing tool that combines established algorithms with rigorous engineering practice to meet explicit performance and robustness targets. The targets come from an empirical study of real open-source projects—the sizes and shapes of their files and of the changes developers actually make—rather than from worst-case analysis alone.

The paper is organized around three research questions, each answered by the correspondingly numbered research answer, RA1–RA3, set in a box at the point in the paper where the measurement supporting it is reported.

- **RQ1 (Scale).** What percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget? Answered by RA1, Section 3.
- **RQ2 (Tool accuracy).** How accurately do current state-of-the-art code-diffing tools recover real-world code changes, measured against human-authored ground-truth mappings of those changes? Answered by RA2, Section 5.
- **RQ3 (Ground-truth ambiguity).** When does a real-world code change have no single correct mapping between the two syntax trees—because several one-to-one mappings are equally correct, or because the true correspondence is not one-to-one at all? Answered by RA3, Section 5.

This paper makes four contributions, one empirical, one system, one dataset, and one evaluation:

- **Empirical study.** An empirical study of 100 real-world open-source repositories, showing that the worst-case inputs tree-diffing algorithms are analyzed against are rare in practice, but that the tail is real and large enough that a production tool must handle inputs beyond what a whole-tree tree-edit-distance computation can process within an interactive budget.
- **System.** CodeDiff itself, a five-phase diffing pipeline built from APTED tree edit distance, scoped to bounded subtree pairs rather than to whole files, together with GumTree's move-recovery heuristic and Myers' sequence diff, engineered as one production tool meeting the targets the empirical study sets.
- **Dataset.** A corpus of 468 real-world code changes in 24 languages, each carrying a human-authored ground-truth AST mapping in which every node is labeled with one of seven operations—and in which a set of interchangeable nodes can be recorded as admitting several equally correct pairings rather than forced to one, which is what makes RQ3 measurable—produced with a purpose-built annotation tool (Section 5). These human solutions are the ground truth every accuracy number in this paper is measured against, and the methodology behind them is reproducible from the project's research repository.
- **Evaluation.** A quantified comparison of four established diffing tools—GumTree, Unix `diff`, difftastic, and diffsitter—against that ground truth, and a measurement of how often that ground truth admits no unique correct mapping, together with instances of a stronger failure in which no one-to-one mapping is correct at all—which together bound what any evaluation scoring a tool against one fixed pairing can report. Separately, a leave-one-out ablation study of CodeDiff's own optional heuristics, showing that a heuristic's published benefit does not automatically transfer into a different pipeline.

2 BACKGROUND

This section introduces the algorithms and tools the rest of the paper depends on: the algorithms CodeDiff builds its pipeline from, and the tools it is evaluated alongside. Section 6 returns to the closest prior work in more depth once the paper's own results are in hand.

2.1 Algorithms

Zhang-Shasha [9] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realizes that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems. Later algorithms reduce this overhead while keeping the same exact result.

APTED [8] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimizes the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower, because most real trees are far from the adversarial shape that bound assumes.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by the Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. This design favors speed over exactness, at the scale real source files require.

Myers' O(ND) algorithm [7] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when

two sequences are similar, exactly the common case for a version-controlled file.

2.2 Tools

difftastic [5] and **diffsitter** [3] are a newer generation of practical, tree-sitter-based diffing tools, the same parsing layer CodeDiff itself is built on. Both prioritize a diff a human reads comfortably over an exact node-to-node mapping: difftastic reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and diffsitter applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars. Neither computes a full tree-edit-distance mapping the way APTED or GumTree do, so neither directly targets the ground-truth-accuracy metric this paper measures against in Section 5—a difference in design goal, not a shortcoming, and the reason we include both as companions in the same problem space rather than as head-to-head competitors on identical terms.

3 PROBLEM SPACE

Formal treatments of the algorithms in Section 2 report worst-case asymptotic complexity: APTED and Zhang-Shasha are both $O(n^3)$ for general trees, where n is tree size. This analysis is necessary, but it is not sufficient for a production tool. GumTree's own documentation states that it targets research use, not production deployment at scale—a reasonable scope for its own goals, but one that leaves open a question every production diff tool must answer:

RQ1 (Scale), stated in Section 1 and restated here: what percentage of real-world source-code changes can a single, whole-tree tree-edit-distance computation—the algorithmic core every exact, syntax-aware diff tool is built on—complete within a one-second budget?

A tool engineered only against a worst-case bound risks two failure modes. It can over-engineer for inputs that never occur in practice. It can under-engineer for a real tail its asymptotic analysis never quantified. Answering RQ1 needs measurement, not further asymptotic analysis.

We measure this directly. We built a corpus of 100 open-source repositories, drawn from the Gentoo Linux distribution's package list and popular repositories on GitHub. We cloned each repository to a depth of 50 commits from every branch tip and parsed every file with Tree-sitter, recording byte size, line count, language, and AST node count per file. The depth limit matters for what follows: it bounds how far back the commit sampling in this section can reach, and a sampled (repository, commit, path) triple stays resolvable only while that commit remains within the depth window of the local clone. The full methodology, and every per-language number, is reproducible from the project's research repository; here, we summarize the numbers that bound what any syntax-aware diff tool has to process.

The corpus has 1,347,490 files across 30 languages. About two-thirds of all files are code. The rest is configuration, data, and documentation (Figure 1). Restricted to code files,

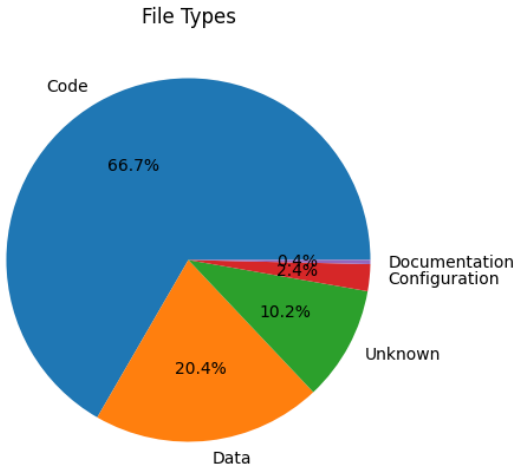

Figure 1: Distribution of file types across the corpus.

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	p99.9	Max
Bytes	1,508	13,945	90,132	464,325	23,949,786
LOC	42	378	2,244	7,818	65,563
AST nodes	224	2,868	18,553	70,828	573,334

Table 1 reports size percentiles in bytes, lines of code, and AST nodes.

The largest file in the corpus has 573,334 AST nodes. Byte size and AST node count have a strong correlation, Pearson $r = 0.8794$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates. Arafat and Riehle report that 47.5% of all commits touch 10 lines or fewer [1], while commits of over 10,000 lines still occur.

Measurement. To answer RQ1's percentage directly, we sampled 2,922 real (before, after) file pairs from single-parent commits in these repositories, stratified per language and per size bucket so that large files are represented rather than drowned out by the far more common small ones. We ran a single whole-tree tree-edit-distance computation on every pair—one APTED implementation, applied to the full before and after trees with no pipeline heuristics of any kind around it—under a hard one-second budget enforced by terminating the computation's process. This implementation includes an optimization that makes it faster than a textbook APTED, so the failure rates below are a lower bound on how often a whole-tree computation misses the budget, not an upper bound.

Every sampled pair was checked to still resolve against the local clones before measurement, and all of them did. This is worth stating because the check has failed before: an earlier sample, drawn against a shallower corpus, had decayed to roughly 41% unresolvable by the time it was re-measured,

and the losses fell on whole repositories rather than spreading evenly across the sample. A measurement over that sample would have reported a healthy-looking rate while quietly answering a question about a different, smaller corpus.

Figure 2 shows the fraction of pairs completed within one second, per size bucket, split by what kind of artifact the file is: code (1,942 pairs across general-purpose programming languages), configuration, data, and markup formats (700 pairs), and scripting languages (280 pairs). The split matters because the three populations behave differently—blended, the headline number would move with how many configuration files a sample happens to contain, independently of anything the algorithm does to source code.

RA1 (Scale)

For source code, a whole-tree tree-edit-distance computation is dependable only for small files, and the transition is abrupt rather than gradual: it completes within one second for 97.9% of code pairs of 10–30 lines, but for only 25.3% of code pairs of 100–300 lines, and never recovers above one third in any larger bucket. Across all sampled code pairs, 54.5% complete in time.

What makes that a design problem rather than a tail problem is where the collapse falls: between two adjacent buckets, neither of which is an unusual size for a source file, and both well inside the range ordinary source files occupy (Table 1). The corpus-wide figure is a sample-weighted aggregate: it deliberately over-represents large files and therefore understates the rate a uniform sample of real commits would show, but the per-bucket rates carry the answer regardless of weighting.

The three artifact categories do not behave alike. Scripting languages complete within the budget 83.9% of the time, well clear of code at 54.5% and configuration, data, and markup formats at 49.0%; the latter two are close to each other and separated by less than six points, so the meaningful split is scripting against the rest rather than a clean three-way ordering. Figure 2 shows the same pattern per bucket: scripting stays above every other category in all seven, while configuration and data fall furthest in the large buckets, despite containing the most repetitive and most regularly structured files in the corpus. We conjecture this reflects tree edit distance being sensitive to tree shape rather than to node count alone, a large flat sequence of similar siblings being an unfavorable shape for it. This measurement does not isolate shape from size, so we state it as a conjecture rather than a result.

The common case is genuinely cheap—most files and most commits are small—but the tail is real: files up to 573,334 AST nodes occur in this corpus, and Arafat and Riehle's data shows commits of over 10,000 lines occur in practice. A production diff tool must be engineered for both regimes, and cannot rely on a whole-tree computation alone for either. Section 4 turns this answer into CodeDiff's concrete design targets.

4 CODEDIFF

The measurements in Section 3 give CodeDiff two concrete, named design targets, stated in place of an asymptotic bound. **Robust**: CodeDiff must handle every file up to the measured maximum, 573,334 AST nodes, with headroom above that number. **Fast**: CodeDiff must compute a diff in under 400 ms for 99.99% of real-world commits, the range that covers the overwhelming majority of commits per Arafat and Riehle's measurement. Section 5 reports how well CodeDiff meets both targets in practice.

CodeDiff matches two ASTs in five phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched. A sixth step closes the mapping, recording the deletion or insertion implied by whatever every phase above left undecided.

Phase 1, hash-based descent. CodeDiff matches subtrees by two hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value. Each variant claims what it can before the next runs. This phase is CodeDiff's own design, motivated directly by Section 3's commit-size data: since most commits touch a small fraction of a file's lines, most of a typical file's AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation's cost on content that never changed.

Phase 2, contextual exact matching. Comments and modifiers (attributes, decorators) that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Both are cheap, high-confidence matches that reduce the work left for the tree-edit-distance phases below. Neither is reachable by phase 1's structural hashing: comments and modifiers are not part of the hashed AST shape, and diagnostic statements are held back deliberately, so that matching one does not fragment a larger match around it.

Phase 3, syntax-aware subtree matching. This phase generalizes three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name structure, such as an `if` block or a loop body, and an overlap matcher pairing import statements that share a base path. Before any of these run, large flat subtrees, such as a macro's token list or a big flat argument list, are pre-matched directly with Myers' algorithm [7], since a flat sequence has no tree structure worth exploiting—exactly the structures that drive Section 3's measured AST-size tail. Each accepted pair is then handed to APTED [8], scoped to that pair alone, to compute its exact internal edit script. This is the only place a tree-edit-distance computation runs at all, and the scoping is what keeps it affordable: APTED sees one accepted container pair at a time, never the whole file.

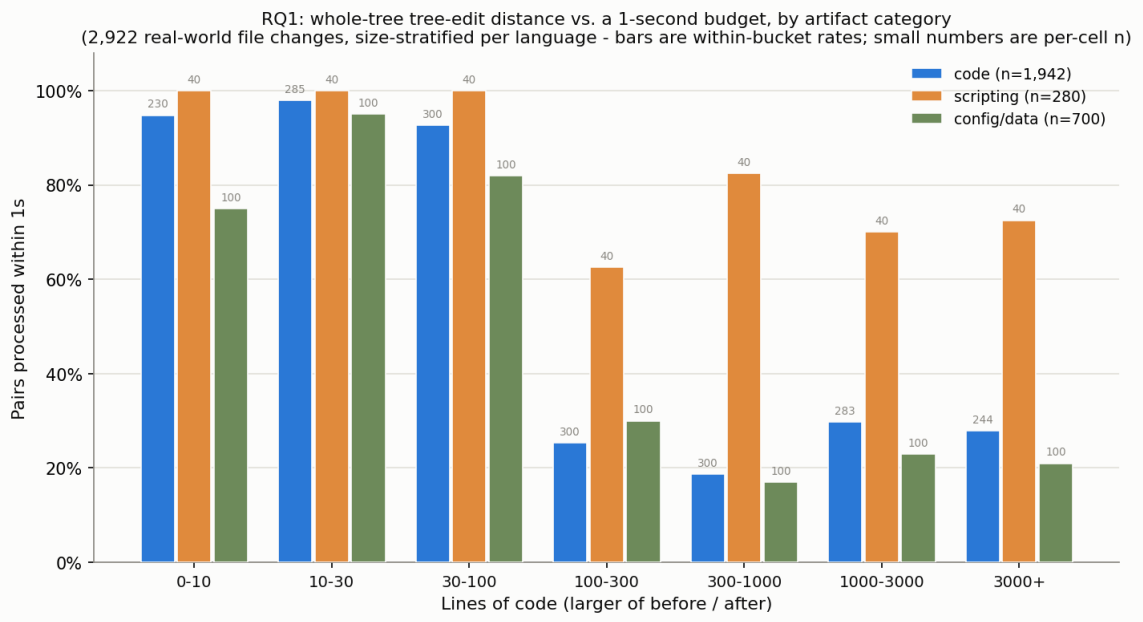

Figure 2: Fraction of sampled file pairs a whole-tree tree-edit-distance computation completes within a one-second budget, per size bucket and artifact category.

Phase 4, residual alignment. Whatever remains unmatched after phases 1 through 3 is resolved by a Myers-LCS alignment [7] over the residual forest, bracketed by two passes of strict bottom-up propagation: a node whose children all match the children of exactly one node on the other side is matched to it, which shrinks the residual the alignment must guess at, and is run again afterward so that pockets the alignment just resolved can still bubble up to their now-complete ancestors.

Earlier versions of CodeDiff instead ran one whole-residual APTED computation here, with the Myers-LCS alignment substituting only when that computation looked too expensive. That design was abandoned: its cost is driven by residual *shape* rather than size, which makes it impossible to gate reliably. Two measured cases make the point—an 11,647-node Vimscript residual took 87 seconds, while a 258,504-node JSON residual took 9.4. No size threshold separates those two, so the exact computation was removed from the whole-file path entirely and survives only in phase 3's scoped form. This is a direct consequence of the size tail in Section 3: an unbounded whole-residual computation cannot meet the 400 ms target at any threshold.

Phase 5, move-detection recovery. The final matching pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree's recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence. It runs last by necessity rather than by preference: every phase above requires some anchor—a hash, a name, a shared matched ancestor—before it will consider a pair at all, and content that moved between two containers that both changed identity has none. Section 5 finds this pass contributes more accuracy than every other optional pass in the pipeline combined.

Zhang-Shasha [9] plays no role in this pipeline as a shipped algorithm. CodeDiff's own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED's result always matches Zhang-Shasha's, bit for bit. This checks the pipeline's most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

5 EVALUATION

Dataset. Every accuracy number in this paper rests on a corpus of 468 fixtures spanning 24 languages. Each fixture pairs a before and after version of a source file, representing a realistic code change—a bug fix, a refactor, an added feature, or a performance-motivated rewrite. Most fixtures are captured directly from a specific commit in a real open-source repository; the rest are curated examples of a common, realistic change pattern. For every fixture, a human annotator used a purpose-built tool, `human_solver`, to walk the before and after syntax trees side by side and label every node with one of seven operations: identical, update, match-but-not-identical, insert, insert-with-children, delete, and delete-with-children. Nodes are addressed by a path of kind and sibling position, not by parser-internal node identity, since that identity is not stable across separate parses of the same file.

Where several candidate nodes are genuinely interchangeable, a single pairing would be an arbitrary choice rather than a fact about the change, so the tool also lets the annotator record a *multi-map group*: a set of N before-side nodes and M after-side nodes in which any consistent pairing is correct, scored as correct as long as a tool realizes $\min(N, M)$ pairs and treats the remainder of the larger side as inserted or deleted. A group is not an expression of annotator uncertainty. It is the annotator's finding that more than one mapping is equally right, and it is what RQ3 measures. This human-authored mapping, groups included, is the ground truth every number in this section is measured against.

This section answers RQ2 and RQ3, stated in Section 1 and restated below, building on RA1's finding that a real, non-negligible tail of large files and commits exists alongside a small common case, and then evaluates CodeDiff itself—the paper's system contribution—against the design targets stated in Section 4.

RQ2 (Tool accuracy). How accurately do current state-of-the-art code-diffing tools recover real-world code changes, measured against human-authored ground-truth mappings of those changes?

RQ3 (Ground-truth ambiguity). When does a real-world code change have no single correct mapping between the two syntax trees—because several one-to-one mappings are equally correct, or because the true correspondence is not one-to-one at all?

Comparison tools. We measure four established tools on the 468-fixture corpus: GumTree (v4.0.0-beta8) [4], Unix `diff`, and the two tree-sitter-based tools from Section 2, difftastic [5] and diffsitter [3]. CodeDiff is deliberately absent from this comparison: RQ2 asks what the state of the art recovers, and that question is answered without reference to the tool this paper introduces. CodeDiff's own accuracy is reported separately, later in this section, against the same ground truth but not as a ranking against these four.

This comparison measures one specific property: agreement with the human-authored ground-truth mapping above. difftastic and diffsitter deliberately optimize for something this metric does not capture, a diff that reads comfortably to a human, not a mapping that reproduces a ground truth line for line, so a lower score here reflects a different design goal, not a defect. GumTree's broader language and backend flexibility likewise stays valuable well beyond what this single corpus measures. One consequence of that flexibility is visible in the numbers below: every generator GumTree offers is included here, and all but two of them are marked "Testing" rather than "Stable" by GumTree's own documentation, so its rate reflects a deliberately wide language scope rather than its performance on the backends it considers ready. Read what follows as a measurement of where each tool's design goals place it on this one metric, not as a ranking of which tool is best.

Line-level agreement. None of the four tools report an AST-node mapping, so we compare them on a common ground: which source lines each marks as touched, against the same human mapping (Table 2, Table 3). Coverage varies widely: diffsitter's compiled-in grammar set matches 303 of the 468 fixtures' languages, GumTree's generator set matches 373, difftastic's 465, and only Unix `diff` covers all 468. Each tool's own row is scored only on its own applicable subset, not padded with unscored fixtures. Because those subsets differ so much, the table also reports every tool on the 260 fixtures all four scored, which holds the fixture set fixed at the cost of over-representing the mainstream languages every tool supports.

Table 2: Line-level mismatch rate against human ground truth, for the four established tools, on each tool's own applicable subset of the 468-fixture corpus and on the 260 fixtures every tool scored.

Tool	Fixtures	Mismatched lines	Rate	Common
Unix `diff`	486	6,732	1.134%	0.786%
diffsitter	303	6,186	1.410%	1.033%
difftastic	465	9,987	1.798%	1.886%
GumTree	373	18,299	4.853%	5.520%

RA2 (Tool accuracy)

Across the four tools (Table 2), line-level mismatch rates against the human mapping range from 1.134% (Unix `diff`) to 4.853% (GumTree). Every tool agrees with the human mapping on the great majority of lines, so what separates them is the small, structurally interesting remainder—and on that remainder no AST-aware tool among the four beats plain line-based `diff`, on either basis. Syntax-awareness alone does not guarantee that a tool recovers a change the way a human reads it.

Table 3 reports the same agreement per fixture rather than pooled over lines, and the two readings weight the corpus differently: a pooled rate is dominated by a handful of very large fixtures, while the buckets weight every change equally. On the pooled rate the four tools span 1.13% to 4.85%; by fixtures mapped with no mismatched line at all they span 50% down to 34%. The table also groups the tools by whether their output carries sub-line detail at all, which is a statement about what each tool reports, not about how it was scored here – every number in this section is line-level.

Two details behind that answer are worth stating. The first is how wide the agreement is: even the highest mismatch rate leaves more than 95% of lines in agreement, so the question that separates tools is a narrow one. The second is that a mismatch rate of this kind presumes there is a single correct mapping to measure against at all, which is the assumption RQ3 turns to next.

Ground-truth ambiguity. Every accuracy number above rests on an assumption worth testing: that for a given change there is one correct mapping to recover. RQ3 asks when that assumption holds, and the corpus shows it failing in two different ways, which we keep apart because they have different consequences.

Table 3: Per-fixture agreement with the human mapping, bucketed. Every number is *line-level* agreement, including for the tools grouped as reporting sub-line detail – that grouping describes what each tool's output carries, not how it was scored. "Perfect" means zero mismatched lines, not a rounded 100%. Each tool is scored on its own applicable subset (n), so percentages, not counts, are comparable across rows.

Tool	n	Perfect	≥99%	95–99%	<95%
Line granularity only					
UNIX diff (baseline)	486	244 (50%)	135 (28%)	60 (12%)	47 (10%)
git (Myers)	486	246 (51%)	138 (28%)	56 (12%)	46 (9%)
git (minimal)	486	246 (51%)	138 (28%)	56 (12%)	46 (9%)
git (patience)	486	245 (50%)	137 (28%)	58 (12%)	46 (9%)
git (histogram)	486	245 (50%)	137 (28%)	58 (12%)	46 (9%)
Reports sub-line detail (not exercised by this line-level metric)					
BDiff (per process)	486	240 (49%)	132 (27%)	66 (14%)	48 (10%)
nvim -d	486	245 (50%)	138 (28%)	57 (12%)	46 (9%)
diffsitter	303	103 (34%)	94 (31%)	62 (20%)	44 (15%)
difftastic	465	255 (55%)	90 (19%)	59 (13%)	61 (13%)
GumTree (binary)	373	181 (49%)	73 (20%)	62 (17%)	57 (15%)

Multiple optimal solutions exist. Several candidate nodes are genuinely interchangeable—one of two identical calls is removed, a member is appended to an expression chain, an argument is added to a list of similar arguments—so more than one one-to-one pairing reproduces the change equally well, and choosing between them is arbitrary rather than correct. A diffing algorithm can still emit a correct answer here; what fails is the assumption that the correct answer is unique.

No one-to-one optimal solution exists. The true correspondence is N:M: several nodes on one side correspond to one or several on the other, all of them at once. Two call sites are refactored into one, a block of comment lines is condensed into a single line, one import is split across two. Here no one-to-one mapping is correct, however the algorithm chooses, and the qualifier matters: not that no optimal solution exists, but that none in the one-to-one form these algorithms produce reaches the true correspondence.

The first is measurable on this corpus, because the annotation tool records it explicitly as a multi-map group (dataset paragraph above). The second is not, and we treat it accordingly below.

One property of the corpus has to be stated before the rate, because it bounds what the rate can mean. The multi-mapping facility was added to the annotation tool partway through corpus construction. 217 fixtures were last annotated before it existed and were never revisited; none of them records an ambiguity, and none of them could, whatever the change actually contained. Table 4 therefore splits the corpus by annotation era rather than reporting one blended rate, which would measure annotation practice as much as the phenomenon.

The middle row is the estimate. Of the 236 fixtures annotated from the start by someone who had multi-mapping available, 36—15.3%—contain at least one place where no single pairing is the correct one. The third row is reported for completeness and excluded from that estimate: those 15 fixtures predate the facility and were reopened afterwards, and a fixture is reopened precisely because something in it needed attention, so their 73.3% rate is selection-biased upward by construction. The corpus-wide 10.0% is a floor rather than a census, for the reason the first row makes explicit.

Table 4: How often the ground truth admits no unique mapping, split by whether the annotator had the multi-mapping facility available. Only the second row is an unbiased estimate; the third is selection-biased and the first is structurally zero.

Annotation era	Fixtures	Ambiguous	Rate
Before the facility existed	217	0	0.0%
Annotated with it available	236	36	15.3%
Predates it, later revisited	15	11	73.3%
Whole corpus (a floor)	468	47	10.0%

The ambiguity is local. Across the 209 groups in the corpus, the median group spans 2 candidate nodes on its larger side and the largest spans 10; exactly 1 extends the ambiguity to whole subtrees rather than to individual nodes. 98.6% of groups have different candidate counts on the two sides, and 55.5% of all groups are the smallest such case, two candidates on one side and one on the other or the reverse. An unequal group usually asks which of N interchangeable nodes is the one that was added or the one that survived—but not always, and the next subsection is about the cases where it asks something else. Ambiguity is also unevenly spread: one fixture alone carries 36 of those groups. The two operations a group can carry split almost evenly, 106 identical against 103 matched-but-not-identical, so ambiguity is not a property of unchanged code alone.

How many fixtures are affected and how much of the ground truth is affected are different quantities, and both are

Table 5: Changes in the corpus whose true correspondence is N:M, found by inspection of annotator commentary rather than by enumeration. "Encoded" is whether the ground truth carries a multi-map group at the site ("Enc."); where it does, that group is the approximation, not the correspondence.

Change	Correspondence	Enc.
License comment condensed	5 comments to 1	yes
Import split in two	1 identifier to 2	yes
Two string literals merged	2 literals to 1	yes
If rewritten as a ternary	2 statements to 1	yes
Two call sites merged	2 statements to 1	no
Two asserts split into six	escape sequences	no

worth stating. Weighted by mapping decisions rather than by fixtures, the groups account for 268 of the 6,786 node pairs the ground truth records outside identical code—3.9%. The two readings are both correct and they answer different questions: a metric aggregated over nodes or lines absorbs an ambiguity of this size almost invisibly, while a metric that asks whether a fixture was mapped exactly right inherits the full 15.3%, because one ambiguous pair is enough to fail a fixture.

No one-to-one optimal solution exists. The second failure is not a matter of degree. An ordered tree edit distance maps each node of one tree to at most one node of the other; that is part of the definition of the edit script it computes, not a limitation of any particular implementation. An N:M correspondence is therefore not in the codomain of any algorithm in this family, GumTree and CodeDiff included. It cannot be found by a better heuristic, because there is nothing in the output language to find it with. The best such an algorithm can do is pick one pair and render the rest as insertions or deletions—which is not an approximation that happens to score badly, but a statement that is false about the change.

We report this as an existence result, not a rate. Nothing in the corpus format marks it: the annotation tool has no way to express N:M either, so an annotator who meets one records the closest one-to-one approximation and writes the real correspondence in prose, in the fixture's test case. The instances in Table 5 were found by reading those comments, not by enumeration, so they are a lower bound of unknown tightness and no denominator is quoted for them.

Four of the six carry a multi-map group sitting exactly at the N:M site, and reading those groups is what shows the encoding is doing double duty. In the condensed license block the group lists five before-side comment nodes against one after-side comment; in the split import, one before-side identifier against two after-side identifiers. None of the four is a set of interchangeable candidates of which one survived. Every member corresponds, and the group's own semantics—realize $\min(N, M)$ pairs, delete or insert the remainder—assert the opposite. The same encoding therefore carries two different phenomena, and only the annotator's prose separates them. The remaining two instances carry no group at all: the correspondence was noted and left unencoded, which is the more honest of the two options available.

This also qualifies a claim made above. Accepting any consistent pairing within a group removes the scoring error for the first phenomenon exactly, because every pairing it admits really is correct. For the second it does not: it scores the leftover deletions as correct when the truth is that those nodes correspond too. The corpus is right about interchangeability and charitable about N:M. Two of the instances in Table 5 postdate the corpus snapshot every other number in this section is measured against, which is why they appear here and in no rate.

RA3 (Ground-truth ambiguity)

A correct AST mapping is not always unique. In 15.3% of the fixtures annotated with multi-mapping available (36 of 236), at least one set of nodes admits several equally correct pairings, with the corpus-wide 10.0% standing as a floor; weighted by mapping decisions rather than fixtures, that is 268 of 6,786 non-identical node pairs, 3.9%. An evaluation that scores a diffing tool against one fixed pairing therefore risks marking correct output wrong on roughly one change in six—whenever the tool picks a different, equally valid pairing—and any accuracy metric defined over a single ground-truth mapping carries that error term whether or not it reports it. A second, rarer failure is categorical rather than statistical: where the true correspondence is N:M, no one-to-one mapping is correct at all, and no algorithm whose output is a one-to-one node mapping can express one. We report instances of that rather than a rate, because neither the corpus format nor the algorithms have a way to represent the thing being counted.

Both rates in Table 4 remain lower bounds for a second reason the corpus cannot correct for: a group exists only where an annotator noticed the ambiguity and recorded it. Nothing detects an ambiguity the annotator resolved without noticing it was a choice. That the count is conservative in this direction is what makes it usable—an over-count would be an argument about annotation discipline, whereas an under-count still establishes that the phenomenon is common enough to affect how accuracy is measured.

The practical consequence is a claim about methodology rather than about any tool. Scoring against a single pairing is the default in this area, and on a corpus like this one it puts a measurable error term into every reported rate: on the affected sixth of changes, a tool that picks a different valid pairing is recorded as wrong for a choice that was never wrong. The corpus in this paper avoids that for the first phenomenon by validating any consistent pairing within a group, as the dataset paragraph above describes, which is why the accuracy numbers reported here do not inherit its error term. For the second it has no such escape, and neither does any corpus built on one-to-one mappings. Reporting a

fixed-pairing metric is still defensible, but the ambiguity rate belongs alongside it.

CodeDiff's own accuracy. The remaining measurements report the paper's system contribution against the same ground truth. They are not part of RA2 or RA3 and are not framed against the four tools above. Measured directly against the human mapping, CodeDiff maps 4,956,544 of 4,959,531 AST nodes correctly across the 468 fixtures, 2,987 mismatches, or 99.94% node-level accuracy. That denominator counts every AST node, including every ancestor of every change up to the root, so it partly reflects how deeply a given grammar nests. Restricted to the nodes that carry text of their own and therefore reach the screen when the diff is rendered, the figure is 99.94%, 2,037 mismatches out of 3,380,690. The two rates agreeing to two decimal places is itself the useful result: CodeDiff's residual errors are not concentrated in invisible structural nodes, so the headline figure is not being flattered by nodes no reviewer ever sees.

Ablation study. Which of CodeDiff's optional passes earn their place is a question about this pipeline, not about the state of the art, so it is reported here rather than as an answer to any of the three research questions. It is still the paper's most transferable result, for the reason the unique-type-matching finding below gives. CodeDiff has four optional heuristic passes, each targeting a specific shape of change and each behind its own flag: move-detection recovery (whole subtrees relocated across a matched boundary, phase 5), bottom-up propagation (a container whose children all match one container on the other side, phase 4), mutual-ancestor pairing (a container sitting above an edit whose descendants all survive inside one corresponding container), and unique type matching (a matched parent's single remaining child of a given kind on each side), the last adapted from GumTree Simple's recovery sub-phase. A leave-one-out ablation study measured each pass's real contribution on the 468-fixture corpus above, by disabling one pass at a time and comparing the resulting mismatch count against a baseline with all four enabled.

Table 6 reports the result at two granularities: every AST node, and only those nodes that carry text of their own and therefore reach the screen when the diff is rendered. The two disagree in a way that matters. Across all nodes, three of the four passes earn their place; restricted to what a reviewer actually sees, only one does. Bottom-up propagation's +318 nodes and mutual-ancestor pairing's +23 are, without exception, structural interior nodes—the containers and wrappers a rendered diff never displays on their own. Unique type matching moves neither column.

On this corpus the move is the one shape whose dedicated heuristic pays for itself in a way a reader would notice: move-detection recovery contributes more accuracy than the other three optional passes combined, by a factor of seven across all nodes, and is the only one of the four that changes the visible-node count at all. A published benefit does not transfer automatically: unique type matching, GumTree Simple's own recovery sub-phase, is correct in isolation yet contributes exactly nothing here, because earlier phases reach its cases first.

Table 6: Ablation study: accuracy change from disabling each optional pass, against a baseline with all four enabled. A positive number means disabling the pass hurt accuracy, more mismatches. Visible nodes are those carrying text of their own, the ones a rendered diff displays. The two passes adapted from GumTree are marked (G).

Pass	Δ all nodes	Δ visible
Move-detection recovery (G)	+2,266	+1,564
Bottom-up propagation	+318	+0
Mutual-ancestor pairing	+23	+0
Unique type matching (G)	+0	+0

That the move earns its pass matches the structural argument of Section 4—a relocated subtree is the one shape ordered tree edit distance cannot express except as a delete plus an insert, so no downstream computation can recover it without a dedicated pass.

Unique type matching is the sharper of the two results, and the reason it contributes nothing is not that it is unsound: instrumentation shows it fires *zero* times across the entire corpus. By the time it runs, CodeDiff's earlier phases have already resolved every case it targets, and a matched parent's leftover unmatched children essentially never land on the "exactly one of this kind on each side" configuration it requires. A heuristic's value is therefore a property of the pipeline it is placed in, not of the technique alone: the same pass that recovers real mappings where it originated can be dead code in a pipeline whose earlier phases are comprehensive enough to reach its cases first. We argue this is the more transferable lesson, and it is only visible by measuring firings and accuracy inside the adopting pipeline, rather than inheriting the original paper's result.

An ablation has one structural limit worth naming, since it bounds what the table above can be read to say: it reports only on heuristics somebody wrote. A shape of change that no pass targets contributes its errors to the baseline and to every ablated configuration alike, so it never appears as a delta in either column. The table ranks the passes that exist; it is silent about the ones that do not.

Production viability. The remaining measurements evaluate CodeDiff itself—speed, then robustness—against the Fast and Robust targets stated in Section 4.

Speed. Table 7 and Figure 3 report wall-clock percentiles over the same fixtures, sorted fastest to slowest by median. We report GumTree twice: once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree's own matching cost. CodeDiff is the fastest AST-aware tool at the median (8.1 ms), but this ordering does not hold across the distribution: diffsitter, whose narrower hand-picked grammar set does markedly less

Table 7: Wall-clock time percentiles, milliseconds, sorted fastest to slowest by median.

Tool	p50	p90	p99
Unix `diff`	2.7	6.3	12.6
CodeDiff	8.1	184.4	2223.9
diffsitter	9.1	65.4	304.4
difftastic	53.4	180.1	2169.4
GumTree (warm JVM)	71.3	747.5	5418.8
GumTree (cold, per-invocation)	610.8	1805.7	13826.2

work, overtakes it at the 90th percentile and is an order of magnitude ahead at the 99th (304.4 ms against 2223.9 ms). CodeDiff's tail is heavy, and Section 4's design explains why: the phases that resolve the common case cheaply do nothing for a large, genuinely dissimilar residual, which is exactly what the slowest fixtures in this corpus are. difftastic's tail is heavier still in aggregate cost and essentially ties CodeDiff at the 99th percentile. Unix `diff` remains fastest at every percentile, since it never parses or compares tree structure.

The two GumTree rows bracket how much of its cost is JVM startup rather than matching. A warm JVM takes its median from 610.8 ms to 71.3 ms, so most of the per-invocation figure is process startup, not GumTree's algorithm. We report both because each answers a different question: the cold number is what a developer running a diff from the command line actually waits for, and the warm number is what GumTree's matching costs once that is amortized away.

These percentiles must be read against the right population. The Fast target of Section 4—400 ms for 99.99% of real-world commits—is a claim about the real distribution of commits, in which 47.5% touch 10 lines or fewer [1]. The fixture corpus is not that distribution: it deliberately concentrates hard, structurally interesting changes, because easy changes exercise nothing worth measuring. CodeDiff's 99th percentile on this corpus (2223.9 ms) is therefore a stress figure on an adversarially weighted sample, not the target metric; its median (8.1 ms) and 90th percentile (184.4 ms) on the same hard sample sit well inside the budget. Validating the 99.99% claim on a uniformly drawn commit sample remains future work, and we state it here as a design target rather than a measured result.

Table 8 reports a companion measurement: how much any single timing run can be trusted, the per-fixture coefficient of variation across repeated measurements of the same fixture. This is what motivated repeating every timing number in this paper across multiple runs rather than trusting a single measurement.

Robustness. We sampled 925 real (repository, commit, file) pairs from real Rust repositories and ran every pair through CodeDiff, up to a 16,000 combined AST-node cap and a 120-second timeout per pair. Of the 925 sampled pairs, 377 were unavailable locally because the repository's history had since been rewritten, not a CodeDiff failure, and 142 exceeded the node cap and were skipped before CodeDiff ever ran. CodeDiff completed all 406 remaining pairs with zero panics and zero timeouts. This is a sampled, bounded-scale result, not a claim over the full 100-repository corpus, and the 142 pairs above the node cap are precisely the inputs the Robust target is about, so they are excluded from the evidence rather than counted in its favor. Within those limits the result is consistent with CodeDiff's stated Robust target in Section 4.

Table 8: Timing noise per series: per-fixture coefficient of variation across repeated measurements of the same fixture (lower means a single run is more trustworthy on its own).

Series	n	Median CoV	p90 CoV
TreeSitter parse (lower bound)	486	5.6%	28.6%
UNIX diff (baseline)	486	8.2%	36.0%
git (Myers)	486	5.1%	32.0%
git (minimal)	486	4.2%	29.5%
git (patience)	486	3.9%	28.1%
git (histogram)	486	4.4%	26.5%
BDiff (per process)	486	4.0%	10.7%
BDiff (warm interpreter)	486	19.7%	39.1%
CodeDiff	486	4.1%	20.6%
GumTree (binary)	373	3.4%	15.0%
GumTree (warm JVM)	373	4.7%	19.3%
diffsitter	303	3.1%	20.9%
difftastic	465	3.0%	16.8%

Summary. Combining established techniques rather than inventing new ones, CodeDiff reaches 99.94% AST-node accuracy against human-verified ground truth, a median well inside its interactive budget, and zero panics or timeouts across the sampled robustness evaluation. Building a production-grade syntax-aware diffing tool did not require a new matching algorithm, only careful combination and measurement of existing ones.

6 RELATED WORK

Tree-diffing tools. GumTree [4] remains the reference point for source-code tree diffing, and CodeDiff's move-recovery phase adapts its recovery-mappings heuristic directly (Section 4). The difference is one of goal: GumTree targets research use, with broad language and backend flexibility, while CodeDiff narrows to a tree-sitter-only frontend and spends the saved generality on production targets—and, as the ablation in Section 5 shows, on validating each adopted heuristic inside its own pipeline rather than assuming the published benefit transfers. difftastic [5] and diffsitter [3] share CodeDiff's parsing layer but optimize for a diff a human reads comfortably rather than for fidelity of the underlying node mapping; RA2 reflects that difference in design goal, not a defect in either tool. GumTree remains the foundation CodeDiff's move-recovery phase builds on directly, and its language and backend flexibility stays valuable well beyond

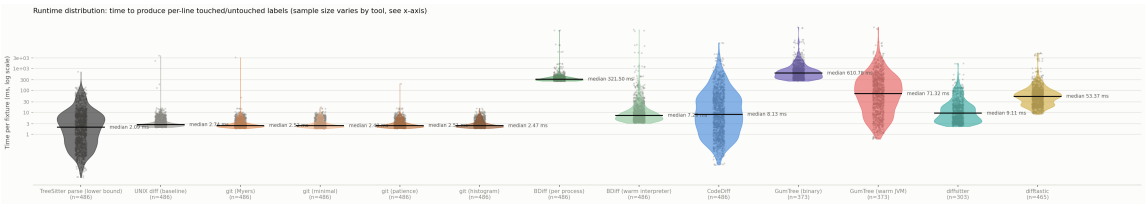

Figure 3: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

what RA2's single corpus measures. Unix `diff`, built on Myers' algorithm [7], remains the baseline every one of these tools must justify itself against—on our corpus it is both the fastest tool measured and, at the line level, more accurate than every pre-existing AST-aware tool compared.

Empirical studies. Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage; Arafat and Riehle [1] measure the commit-size distribution of open-source software, the distribution CodeDiff's Fast target is stated against. This paper's empirical study differs from these in purpose rather than method: it measures file- and AST-size distributions specifically to set, and then evaluate, the engineering targets of a diffing tool.

7 CONCLUSION AND FUTURE WORK

None of CodeDiff's matching algorithms are new. Its contribution is combining APTED's exact tree edit distance, GumTree's move-recovery heuristic, and Myers' sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case, and validating every optional piece of that combination by measurement instead of by assumption. On the corpus in this paper, that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree's language and backend flexibility, difftastic's and diffsitter's speed and readability, Unix `diff`'s universality. The ablation result in Section 5 is the more general, transferable point: a heuristic's value is a property of the pipeline and corpus it runs in, not only of the paper that introduced it. A production tool must measure that value directly, rather than assume it carries over.

The same discipline applies to the exact algorithm at the pipeline's core. CodeDiff began by running one whole-residual APTED computation per file, exactly as its asymptotic analysis invites, and removed it after measurement showed its cost tracks residual shape rather than residual size closely enough that no size threshold can gate it safely. What survives is APTED scoped to individual container pairs, where the same exactness is affordable. An exact algorithm is not automatically the right one to run at whole-file scale, and only measurement distinguishes the two regimes.

Based on the lessons reported in this paper, we recommend the following to builders of syntax-aware developer tooling:

- Measure every adopted heuristic inside your own pipeline, against your own corpus, and count how often it fires as well as what it changes: one heuristic adopted here contributes more than all the others combined, while another, correct and unit-tested in isolation, never fires at all.
- Engineer against the measured distribution of real inputs, not against the worst-case bound alone: the common case and the tail require different machinery, and both occur in practice.
- Scope an exact algorithm to the inputs it can serve within budget rather than abandoning or over-applying it: the same computation that is unaffordable over a whole file is the right tool for a bounded subtree pair.
- Keep an exact algorithm as an independent test oracle even when the shipped path is heuristic: CodeDiff fuzz-checks its APTED implementation against an independently implemented Zhang-Shasha on thousands of random trees.

Future work includes widening the robustness corpus beyond Rust, extending the accuracy comparison to more languages GumTree also supports, and re-running the ablation study as new heuristic passes are added to the pipeline. RA3 points at the clearest methodological direction: the corpus establishes that ambiguity is common, but it establishes it only where an annotator noticed and recorded it. Detecting candidate ambiguity automatically, and re-annotating the 217 fixtures that predate the multi-mapping facility, would turn a floor into a census and let the rate be measured on a corpus rather than on a corpus's annotated half. The $N{:}M$ half needs more than that: it needs a ground-truth format able to express a correspondence that no one-to-one mapping represents, before its prevalence can be measured at all rather than found by reading.

REFERENCES

[1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. https://doi.org/10.1109/HICSS.2009.421

[2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. https://doi.org/10.1145/2568225.2568295

[3] Afnan Enayet. 2020. diffsitter: A Tree-Sitter-Based AST Difftool for Meaningful Diffs. https://github.com/afnanenayet/diffsitter. Accessed 2026-07-31.
[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. https://doi.org/10.1145/2642937.2642982
[5] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. https://github.com/Wilfred/difftastic. Accessed 2026-07-31.
[6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.
[7] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. https://doi.org/10.1007/BF01840446
[8] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. https://doi.org/10.1145/2699485
[9] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. https://doi.org/10.1137/0218082